Operating Systems

Abdelrahman Hosny

2026

Faculty of Computers and Information, Assiut University

Contents

Announcements

- **Project 1** is released
  - Deliver by: Friday April 10th, 2026.
  - There is a question that you need to provide an answer to.
  - Get a submission ID. Discuss it with your TA on the week of delivering the project.
  - Submit as many times as you want. Only use one submission ID with the TA.
  - Note: submission script does fingerprint your submission and the build env.

    ⇒ Cheating will result in all submitters getting a zero grade.

Announcements

- **Midterm**
  - Tests your understanding of:
    - lectures 0 to 4 (this class)
    - labs 0, 1 and 2.
  - 25 questions MCQ
  - 20 pts
  - 10 Sample questions will be released on April 2nd.
    ⇒ might or might not show up in your exam

This Lecture

System calls

What is a system call?

A system call is a public function implemented by the operating system (OS) that can be used within a program. They serve as the primary interface for programs to request services from the OS kernel.

Core functionality

1. **Privileged Tasks:** System calls are used to perform tasks that regular programs are not permitted to do on their own, such as communicating directly with hardware (e.g., calling readSector).
2. **Mode Switching:** When a system call is executed, the CPU switches from user mode (where regular programs run) to kernel mode (where the OS has full control). It switches back to user mode once the call is complete.
3. **Security and Isolation:** By having the OS kernel run the code for these calls, the system isolates critical interactions from potentially harmful or buggy user programs.

Filesystems - System calls

While many high-level programming languages provide abstract methods for tasks like file interaction (e.g., C++ streams), these methods are ultimately built upon low-level system calls.

Common examples for interacting with files include:

1. `open()`: Opens a file and returns a file descriptor.
2. `close()`: Closes an open file descriptor to preserve system resources.
3. `read()`: Reads a specific number of bytes from a file into a memory buffer.
4. `write()`: Writes bytes from a memory buffer into an open file.

Filesystems - System calls

While many high-level programming languages provide abstract methods for tasks like file interaction (e.g., C++ streams), these methods are ultimately built upon low-level system calls.

Common examples for interacting with files include:

1. `open()`: Opens a file and returns a file descriptor.
2. `close()`: Closes an open file descriptor to preserve system resources.
3. `read()`: Reads a specific number of bytes from a file into a memory buffer.
4. `write()`: Writes bytes from a memory buffer into an open file.

Filesystems - System calls

```c
// Open an existing file for reading only and return a file descriptor
int fd = open("example.txt", O_RDONLY);

// If open returns -1, something went wrong (like the file not existing)
if (fd == -1) {
    // handle error
}
```

Filesystems - System calls

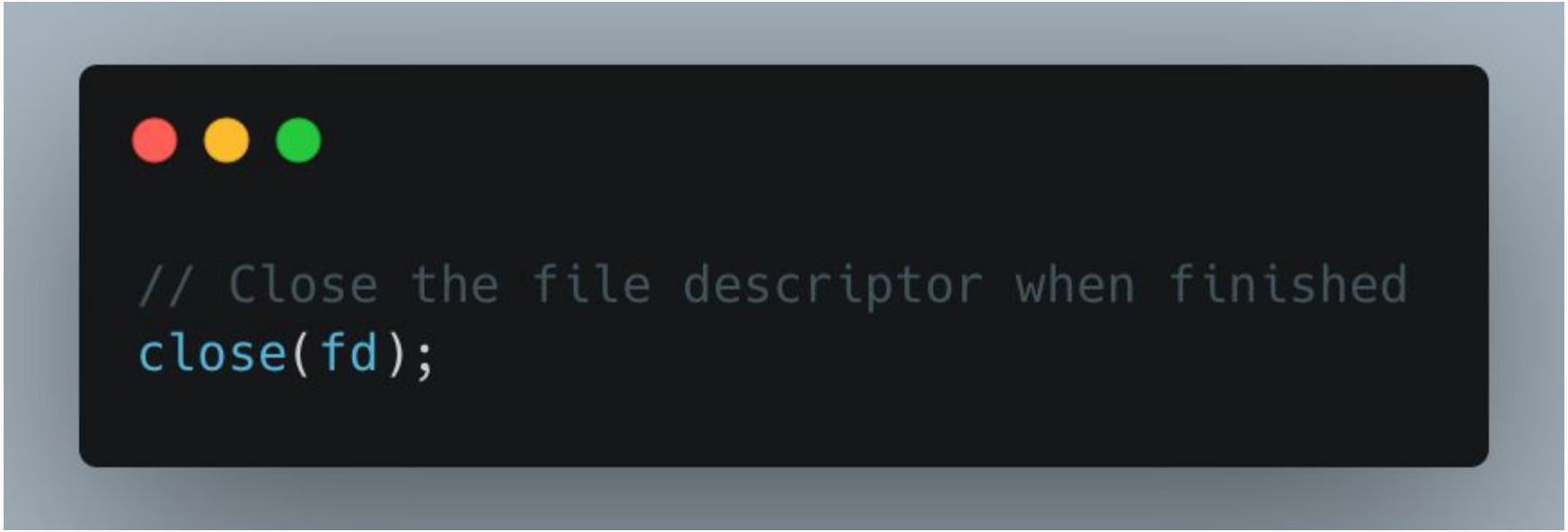
```
char buffer[64];
// Attempt to read 64 bytes into the buffer
ssize_t bytesRead = read(fd, buffer, sizeof(buffer));

// bytesRead will be 0 if you've reached the end of the file [cite: 300]
if (bytesRead == 0) {
    // EOF reached
}
```

Filesystems - System calls

```
const char *msg = "CS111";
// Write 5 bytes to the file associated with fd
ssize_t bytesWritten = write(fd, msg, 5);
```

Filesystems - System calls



What is a file descriptor?

A file descriptor (FD) is a unique, non-negative integer assigned by the operating system to represent an instance of an open file within a specific program.

A file descriptor is **a process-specific** handle used by your program.

What does it relate to inode?

An inode is a filesystem-level structure that represents the actual file on disk. This is what the OS manages.

File descriptors vs. inodes

- **Mapping:** When you call `open()`, the OS finds the **inode** for the requested path and creates a new file descriptor that eventually points to that inode.
- **Multiple Descriptors, One Inode:** You can have multiple file descriptors pointing to the same file (and thus the same inode). Each descriptor maintains its own separate "location" or offset in the file.
- **System Calls:** System calls like `read()` and `write()` bridge these layers: they take a file descriptor from the user program, look up the current offset, and then use the inode information to determine which physical disk blocks to actually access.

File Descriptors and I/O

What if we could use the same code that writes to a file to also write output to the terminal? Or write to a network connection?

What if we could use the same code that reads from a file to read input from the user? Or read from a network connection?

File descriptors let us do this! A file descriptor can represent more than a file – it is a number representing a currently-open resource. That resource could be a file, or something that behaves like a file.

File Descriptors and I/O

- When you connect to the network, you get back a file descriptor. You can read/write with it to read/write on the network!
- There are special reserved file descriptor numbers 0,1,2.
  - Reading from 0 reads input from the terminal
  - Writing to 1 writes to the terminal STDOUT
  - Writing to 2 writes to the terminal STDERR

File Descriptors and I/O

There are 3 special file descriptors provided by default to each program:

- 0: standard input (user input from the terminal) - STDIN_FILENO
- 1: standard output (output to the terminal) - STDOUT_FILENO
- 2: standard error (error output to the terminal) - STDERR_FILENO

These aren't really files. However, they are set up to behave just like they are.

e.g. `read(0, buf, nbytes)` gets input from the user!

Programs always assume that 0,1,2 represent STDIN/STDOUT/STDERR.

e.g. `cin` in C++ is essentially a nicer version of `read(0, buf, nbytes)`!

File Descriptors and I/O

```c
static const int kDefaultPermissions = 0644;
int main(int argc, char *argv[]) {
    int sourceFD = open(argv[1], O_RDONLY);
    int destinationFD = open(argv[2], O_WRONLY | O_CREAT | O_EXCL, kDefaultPermissions);

    copyContents(sourceFD, destinationFD);

    close(sourceFD);
    close(destinationFD);

    return 0;
}
```

What is the smallest 1 line change/hack we could make to this code to make it print the contents of the source file to the terminal instead of copying it to the destination file?

Processes

Terminologies

Program: code you write to execute tasks

Process: an instance of your program running; consists of program and execution state.

Key idea: multiple processes can run the same program

```c
int main(int argc, char *argv[]) {
    printf("Hello, world!\n");
    printf("Goodbye!\n");
    return 0;
}
```

Multiprocessing

Your computer runs many processes simultaneously - even with just 1 processor core (how?)

- "simultaneously" = switch between them so fast humans don't notice
- Your program thinks it's the only thing running
- OS schedules tasks - who gets to run when; and who is misbehaving and should be killed
- Caveat: multicore computers can truly multitask

An inside look

When you run a program from the terminal, it runs in a new process.

- The OS gives each process a unique "process ID" number (PID)
- PIDs are useful once we start managing multiple processes
- `getpid()` returns the PID of the current process (`pid_t` is a numeric type)

```c
#include <stdio.h>
#include <unistd.h>
int main(int argc, char *argv[]) {
    pid_t myPid = getpid();
    printf("My process ID is %d\n", myPid);
    return 0;
}
```

Introducing `fork`

`fork` is a system call that

- creates a second process which is a clone of the first.

```c
int main(int argc, char *argv[]) {
    printf("Hello, world!\n");
    fork();
    printf("Goodbye!\n");
    return 0;
}
```

Introducing `fork`

`fork()` creates a second process that is a clone of the first:

- parent (original) process forks off a child (new) process
- The child starts execution on the next program instruction. The parent continues execution with the next program instruction. The order from now on is up to the OS!
- `fork()` is called once, but returns twice (why?)

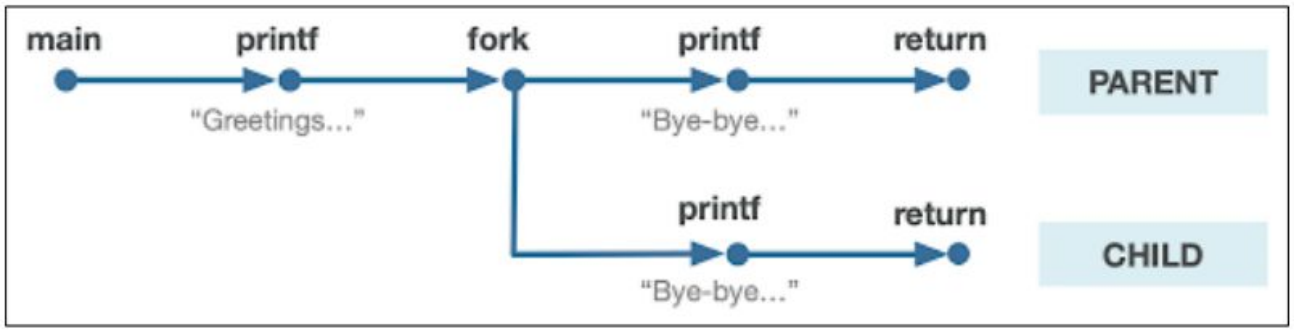


Illustration courtesy of Roz Cyrus.

How do I know if I’m the parent or the child? And why bother?

Any questions?
No stupid question!

Let's discuss any doubts or concerns.
If something comes up later, contact abdelrahman@aun.edu.eg